

ELEC 442

Lab 1

LEARNING GOALS

Forward Kinematics

- *Spatial Mapping Principles*: Discover how geometric constraints reduce 3D spatial transformations down to the four unique parameters of the Denavit-Hartenberg convention.
- *Homogeneous Representations*: Grasp how 4×4 homogenous transformation matrices unify rotation and translation to seamlessly map configuration space to task space.
- *Hardware Boundaries*: Appreciate the physical and software limits of robotic manipulators, including joint limits and workspace restriction volumes.

Velocity Kinematics

- *Differential Kinematics*: Observe the role of the Jacobian and its inverse in mapping rates of change between joint variables and end-effector motion.
- *Limitations of Numerical Integration*: Recognize why digital systems suffer from truncation and round-off error, and how cumulative drift compromises open-loop control.
- *Continuity in Motion Planning*: Evaluate how boundary conditions shape trajectory profiles, and understand why continuity in lower-order derivatives (position, velocity) can still result in harsh higher-order discontinuities (jerk).

1 Forward Kinematics Introduction

Welcome to the first lab! Over the course of this term, we hope to provide an environment where you can apply the principles behind robotic manipulators that you learned in class. This lab provides instructions for setting up the QArm and introduces exercises that apply forward kinematics.

1.1 The QArm

Our instrument of choice for this term is the Quanser QArm, a four-degree-of-freedom manipulator that you will work with in teams of two or three. The four joints are the base, the shoulder, the elbow, and the wrist. You can also open and close the gripper. Lastly, a RealSense D415 RGB and depth camera is included, which will be used later in the term for computer vision applications.

NOTE: Joint angles as an array/list will always be given or expected to be recieved in the order: base, shoulder, elbow, wrist.



Figure 1: Quanser QArm - <https://www.quanser.com/products/qarm/>

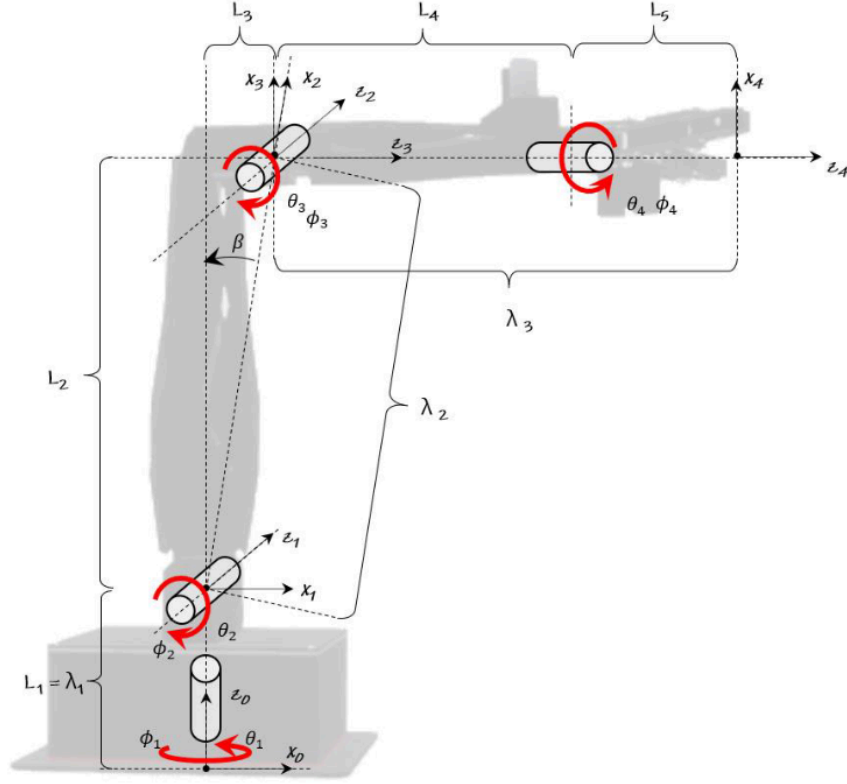
Joint Specifications									
#	Item		Base	Shoulder	Elbow	Wrist	Units	Gripper	Units
1	Joint range	Min. Max	-170 +170	-85 +85	-95 +75	-160 +160	degrees	0 100	%
2	Max. speed		$\pm\pi/2$	$\pm\pi/2$	$\pm\pi/2$	$\pm\pi/2$	rad/s	± 100	%/s
3	Max. acceleration		$\pm\pi/3$	$\pm\pi/3$	$\pm\pi/3$	$\pm\pi/3$	rad/s/s	± 400	%/s/s
4	Current	Instantaneous Continuous	4.4 1.1	8.8 2.2	4.4 1.1	2.3 0.6	Amps	1.4 0.35	Amps
5	Torque	Instantaneous Continuous	10.6 2.65	21.2 5.3	10.6 2.65	4.1 1.0	Nm	1.9 0.5	Nm

Figure 2: Quanser QArm joint limits

2 Denavit–Hartenberg Convention

In a general three-dimensional space, describing the relative position and orientation between two coordinate frames requires six independent parameters: three for translation and three for rotation. However, by imposing specific geometric constraints on how we attach coordinate frames to robot links, the Denavit–Hartenberg (DH) convention reduces this mapping to just four parameters per joint. The transformation from frame $i - 1$ to frame i is uniquely defined by executing the following four operations in sequence:

- θ_i : Rotation about $\underline{k}_{i-1}$ to align the $\underline{i}_{i-1}$ direction with the $\underline{i}_i$ direction.
- d_i : Translation along $\underline{k}_{i-1}$ to align the $\underline{i}_{i-1}$ and $\underline{i}_i$ axes.
- a_i : Translation along $\underline{i}_i$ to complete the shift to the new origin.
- α_i : Rotation about $\underline{i}_i$ to align the $\underline{k}_{i-1}$ axis with the $\underline{k}_i$ axis.



Parameter	Value
L_1	0.14 m
L_2	0.35 m
L_3	0.05 m

Parameter	Value
L_4	0.25 m
L_5	0.15 m
β	8.13°

Figure 3: QArm dimensions and values

Exercise 1:

Fill out the following DH table using the QArm values

Link	θ_i	d_i	a_i	α_i
1				
2				
3				
4				

3 Forward Kinematics

Forward kinematics allows us to determine the end-effector's pose (position and orientation) in task space from a given set of joint coordinates in configuration space. To compute this efficiently, we use 4×4 homogeneous transformation matrices. These matrices consolidate a 3×3 rotation matrix and a 3×1 translation vector into a single operator, applying both rotation and translation simultaneously through a single matrix multiplication. A general homogeneous transformation matrix, T , is defined as:

$$T = \begin{pmatrix} \mathbf{R} & \underline{o} \\ \underline{0}_{1 \times 3} & 1 \end{pmatrix} = \begin{pmatrix} r_{11} & r_{12} & r_{13} & x \\ r_{21} & r_{22} & r_{23} & y \\ r_{31} & r_{32} & r_{33} & z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Where:

- $\mathbf{R}$ in $\mathbb{R}^{3 \times 3}$ is the orthogonal rotation matrix representing the orientation of the new frame relative to the reference frame.
- $\underline{o}$ in $\mathbb{R}^{3 \times 1}$ is the translation vector representing the position coordinates (x, y, z) of the new frame's origin expressed in the reference frame.
- $\underline{0}_{1 \times 3}$ is a row vector of zeros mapping the perspective transformation.

For your convenience, the following is provided, however review your class notes for more information:

$$\begin{pmatrix} \underline{C}_i & \underline{o}_i \\ 0^T & 1 \end{pmatrix} = \begin{pmatrix} \underline{C}_{i-1} & \underline{o}_{i-1} \\ 0^T & 1 \end{pmatrix} \begin{pmatrix} e^{\theta_i k \times} & 0 \\ 0^T & 1 \end{pmatrix} \begin{pmatrix} I & d_i k \\ 0^T & 1 \end{pmatrix} \begin{pmatrix} I & a_i k \\ 0^T & 1 \end{pmatrix} \begin{pmatrix} e^{\alpha_i i \times} & 0 \\ 0^T & 1 \end{pmatrix} = \begin{pmatrix} \underline{C}_{i-1} & \underline{o}_{i-1} \\ 0^T & 1 \end{pmatrix} {}^{i-1}T_i$$

Another way to represent ${}^{i-1}T_i$:

$${}^{i-1}T_i = \begin{pmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Exercise 2: Using your DH table, calculate the full transformation matrix from frame $\{0\}$ to $\{4\}$ (0T_4); feel free to use a calculator or computer. Keep this recorded, as we will use it later.

Tip: Find the intermediate transformation matrices, then multiply them together. We recommend implementing a function such as: `find_matrix_DH(theta, d, a, alpha)`.

4 Experiment Setup

Figure out how we're setting it up:

- I'm thinking: github organizations (in the style of classrooms) for software
- The hardware... hopefully I'll be mostly just turning the arm on
- Figure out position and mounting to know the safety and keep in area

4.1 Safety Warnings

The QArm's workspace will be limited to a [Certain size restricted by a script **Let's see how the arms are mounted for this**] volume to protect the robot, surrounding equipment, and users.

4.2 Software Setup

[Insert instructions for downloading the code and setting up the environment. Note: If using fixed lab computers, specify which files to open.]

4.3 Hardware (QArm) Setup

Please handle the QArm with care. [Insert any additional hardware setup steps here.]

5 Forward Kinematics Experiment

5.1 Implementing Forward Kinematics

Using the joint positions and your derived homogeneous transformation matrix, your next goal is to implement a forward kinematics function.

```
location, rotation = forward_kin(phi)
```

Role	Name	Dimension	Description
input	phi	4	Joint angles in order from base to wrist
output	location	3	Location of the {4} origin in the {0} coordinate system
	rotation	3×3	Representing rotation from {4} (gripper tip) to {0} (base) frame

Tip: Both the location vector and rotation matrix can be extracted from the same homogeneous transformation matrix.

Exercise 3: Test your function using the following inputs and expected outputs:

Input	Location [x, y, z]	Rotation Matrix
[0, 0, 0, 0]	[0.45, 0.0, 0.49]	$\begin{pmatrix} 0.0 & 0.0 & 1.0 \\ 0.0 & -1.0 & 0.0 \\ 1.0 & 0.0 & 0.0 \end{pmatrix}$
[30, 55, -60, 0]	[-0.08, 0.5, 0.86]	$\begin{pmatrix} -0.08 & 0.5 & 0.86 \\ -0.04 & -0.87 & 0.5 \\ 1.0 & 0.0 & 0.09 \end{pmatrix}$

Debug any issues that arise, record your results for the following inputs:

Input	Location [x, y, z]	Rotation Matrix
[0, -11, -75, 0]		
[1, 2, 3, 4]		
[1, 2, 3, 4]		
[1, 2, 3, 4]		

5.2 Pick and Place; Keyboard Controls

Congrats! You are just about to start a new position! Unfortunately, it was another terrible year for the job market, so instead of engineering, you decided to join the prestigious Museum of Impossibly Ridiculous and Fragile Objects (MIRFO) as an assistant night shift curator. MIRFO is filled with objects that make no sense, and worst (or best) of all: everything is just a breeze or a mistimed observation away from falling apart.

Your very first task is to put the finishing touches on the brand-new “Ming Dynasty Porcelain Rubber Ducky” exhibit. Due to frustrating insurance requirements, humans are not allowed near the artifacts, so it is up to you and your trusty robotic arm to position the ducks into place for the grand opening tomorrow morning! Be very careful—the artifact is priceless, and one wrong move from your manipulator can quickly turn it into the “Ming Dynasty Pile of Shards.”

Open `QArm_keyboard_control.py`. When executed, once focused on the pygame window, this script will allow you to manually control the position of the robot using your keyboard. Experiment with moving the arm around to get a feel for its workspace. The Pygame window will also display the live joint angles.

Use number keys [1] through [4] to select any of the 4 joints, then use the up and down arrow keys to move the joint in either direction. The home position will be approximately $(x, y, z) = (0.45, 0, 0.49)$ meters.

⚠ WARNING: To prevent injuries and damage to the arm, it will be restricted to [restriction volume]; any attempt to leave this will cause the arm to lock in place. To reset, simply end and rerun the script; the arm should home then leave you in control again.

There seems to be a bug in the Quanser library where they try to get three outputs from the `forward_kin` function when using the keyboard controls

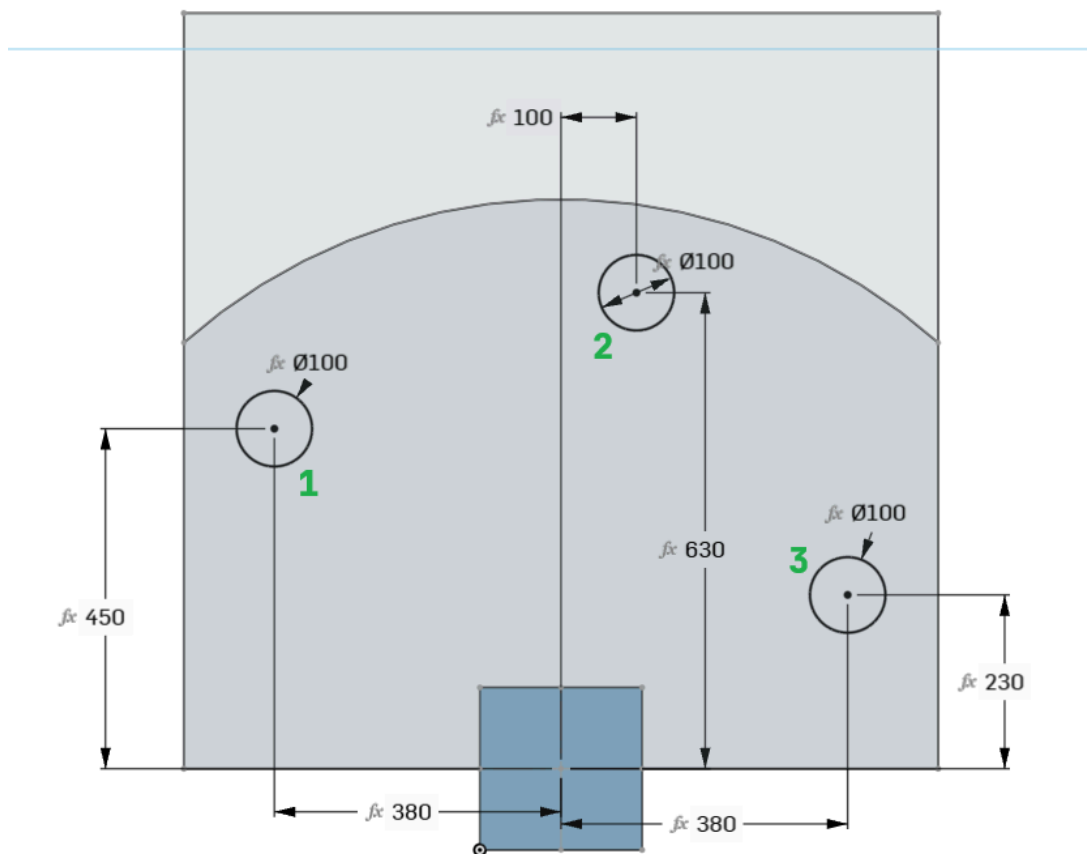


Figure 4: Pick and Place: Zone Diagram

Exercise 4:

Place 2 ducks in zone 1. Manually move one duck into zone 2 and 3. Fill out joint positions at each point of interest as you go:

		Position			
		home	1	2	3
Joint	base				
	shoulder				
	elbow				
	wrist				

5.3 When Things Go Wrong

Explore what happens if we “artificially” introduce some error in accuracy or precision. This can manifest in several ways:

- Joint angle error due to incorrect measurement or backlash/slip
- Link measurement error due to manufacturing issues or material strain
- Signal or sensor noise

Exercise 5:

Calculate and record the task space positional error (magnitude) in several poses when we purposefully introduce error using your forward kinematics implementation.

Pose_1 = [0, 0, 0, 0] (home)

Pose_2 = [0, 55, -60, 0] (extended)

Pose_3 = [0, -80, 70, 0] (retracted)

Error Type	Pose 1 Positional Error	Pose 2 Positional Error	Pose 3 Positional Error
Shoulder joint + 10°			
Elbow joint + 10°			
L2 length + 5cm			
L4 length + 5cm			

Question: Identify which joint and link errors cause the largest overall positional discrepancy. Why does an error at one specific joint or link scale differently than another? Based on this behavior, what is one additional distinct pose you would test to explore the absolute worst-case positional error?

LEARNING GOALS

Forward Kinematics

- ✓ *Spatial Mapping Principles*: Discover how geometric constraints reduce 3D spatial transformations down to the four unique parameters of the Denavit-Hartenberg convention.
- ✓ *Homogeneous Representations*: Grasp how 4×4 homogenous transformation matrices unify rotation and translation to seamlessly map configuration space to task space.
- ✓ *Hardware Boundaries*: Appreciate the physical and software limits of robotic manipulators, including joint limits and workspace restriction volumes.

Velocity Kinematics

- *Differential Kinematics*: Observe the role of the Jacobian and its inverse in mapping rates of change between joint variables and end-effector motion.
- *Limitations of Numerical Integration*: Recognize why digital systems suffer from truncation and round-off error, and how cumulative drift compromises open-loop control.
- *Continuity in Motion Planning*: Evaluate how boundary conditions shape trajectory profiles, and understand why continuity in lower-order derivatives (position, velocity) can still result in harsh higher-order discontinuities (jerk).

6 Velocity Kinematics Introduction

6.1 Motivation

One of the most powerful operators in robotics is the Jacobian matrix. While the primary focus of this lab is utilizing the Jacobian for trajectory generation, it is also fundamentally used to map task-space forces to joint torques, resolve kinematic redundancy, and identify kinematic singularities.

6.2 Theory

6.2.1 Generating Joint Trajectories

The Jacobian relates joint velocities to end-effector linear and angular velocities through the following relationship:

$$\dot{\underline{q}} = \mathbf{J}^{-1}(\underline{q})\underline{v}$$

Where $\dot{\underline{q}}$ represents the joint velocities, $\mathbf{J}^{-1}(\underline{q})$ is the inverse Jacobian matrix dependent on the current joint configurations $\underline{q}$, and $\underline{v}$ is the desired task-space velocity vector.

Using the robot's current position as an initial condition, we can use the inverse Jacobian and a desired task-space velocity vector to numerically integrate and generate a continuous joint trajectory.

For example, this can be implemented in Python as follows:

```
dt = current_time - previous_time
d_joint = J_inv * d_trajectory
joint_pos += d_joint * dt
```

6.2.2 Creating a Trajectory

When designing a trajectory for the manipulator to follow, several critical factors must be considered:

- Must the end-effector pass through specific via-points or targets?
- Are there physical obstacles in the workspace to avoid?
- What are the operational limits on the joint motor torques?
- What are the geometric constraints of the robot's reachable workspace?

While numerous methods exist to smoothly transition from one task-space position to another, a common and straightforward approach is to utilize a cubic spline. A cubic polynomial profile provides four degrees of freedom (coefficients). By specifying four boundary conditions—typically the initial and final positions,

along with setting the initial and final velocities to zero ($\underline{v} = 0$)—we can generate a trajectory with continuous position and velocity profiles.

By evaluating this cubic spline position function at each time step, we can also actively compensate for numerical integration drift. This is achieved by comparing the robot's calculated position against the desired spline target and introducing a feedback correction factor.

For example, this tracking and correction loop can be implemented in Python as follows:

```
v_cmd = spline_velocity(t) + Kp*(spline(t)-location)
```

7 Analytic Jacobian

You are likely familiar with the **Geometric Jacobian** ($J_G \in \mathbb{R}^{6 \times n}$) from class lectures. The geometric Jacobian maps the joint velocities directly to the 3 linear velocity components and the 3 angular velocity components of the end-effector framework:

$$J_i = \begin{cases} \begin{pmatrix} \underline{k}_{i-1} \times (\underline{o}_n - \underline{o}_{i-1}) \\ \underline{k}_{i-1} \end{pmatrix} & \text{if joint } i \text{ is revolute} \\ \begin{pmatrix} \underline{k}_{i-1} \\ \underline{0} \end{pmatrix} & \text{if joint } i \text{ is prismatic} \end{cases}$$

The **Analytic Jacobian** (J_A), on the other hand, relates the joint velocities directly to a designated set of chosen *task space variables*, $\underline{x}$. For our upcoming lab objectives, we will specify our task space using 4 coordinates: the 3 linear coordinate positions (p_x, p_y, p_z) along with a single wrist orientation angle (φ). In this lab, we will be using the analytic Jacobian for velocity kinematics as it's generally much more convenient to work with.

By employing the multivariate chain rule to evaluate the instantaneous rate of change for these specific variables over time:

$$\frac{d\underline{x}}{dt} = \frac{\partial \underline{x}}{\partial \underline{\theta}} \frac{d\underline{\theta}}{dt} \rightarrow \dot{\underline{x}} = J_A(\underline{\theta}) \dot{\underline{\theta}}$$

This formulation yields the following explicit matrix structure for our analytic Jacobian:

$$J_A(\underline{\theta}) = \begin{pmatrix} \frac{\partial p_x}{\partial \theta_1} & \frac{\partial p_x}{\partial \theta_2} & \frac{\partial p_x}{\partial \theta_3} & \frac{\partial p_x}{\partial \theta_4} \\ \frac{\partial p_y}{\partial \theta_1} & \frac{\partial p_y}{\partial \theta_2} & \frac{\partial p_y}{\partial \theta_3} & \frac{\partial p_y}{\partial \theta_4} \\ \frac{\partial p_z}{\partial \theta_1} & \frac{\partial p_z}{\partial \theta_2} & \frac{\partial p_z}{\partial \theta_3} & \frac{\partial p_z}{\partial \theta_4} \\ \frac{\partial \varphi}{\partial \theta_1} & \frac{\partial \varphi}{\partial \theta_2} & \frac{\partial \varphi}{\partial \theta_3} & \frac{\partial \varphi}{\partial \theta_4} \end{pmatrix}$$

Implementation Highlights & Trade-offs

- *The Square Matrix Property:* By purposefully reducing our operational task space to exactly 4 variables matching our 4-DOF manipulator configuration, J_A becomes a clean 4×4 square matrix. This enables standard matrix inversion (J_A^{-1}) for direct inverse kinematics trajectories, bypassing the need for pseudo-inverses.
- *The Representation Constraint:* Ditching the full 3D rotation representation means we forfeit the ability to explicitly extract the absolute physical angular velocity vector ($\boldsymbol{\omega}$). However, because our lab focus centers tightly on targeted linear pathways and a single, predictable orientation, this compromise is well worth it.

8 Velocity Kinematics Experiment

8.1 Tasks

8.1.1 Visualize the purpose

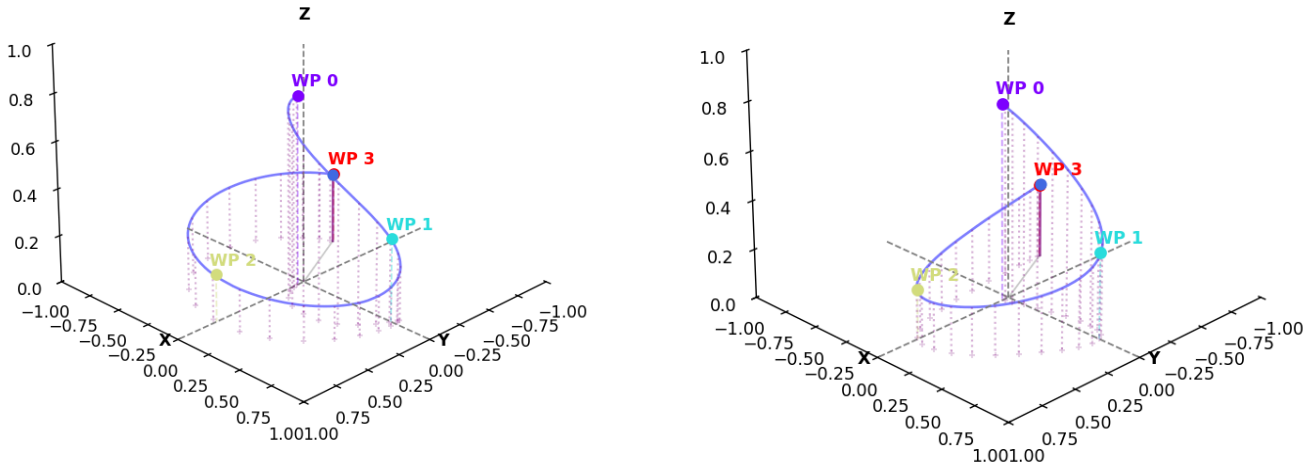


Figure 5: Joint space (left) vs task space (right) interpolation

Open and run `QArm_traj_demo.py`. This script moves the manipulator between the coordinates specified in `trajectory_points` using cubic spline interpolation in either joint space or task space. Figure 5 illustrates what the final output may look like; however, it is highly recommended to run the script yourself and observe the motion over time and from various angles. Also feel free to adjust the points by editing the `trajectory_points` list (note: trajectories that violate joint or workspace limits will terminate the script).

8.1.2 Trajectory Generation

Welcome back to the Museum of Impossibly Ridiculous and Fragile Objects (MIRFO)! Your work on the Ming Dynasty Porcelain Rubber Ducky exhibit was fantastic, and it has promoted you to night-shift curator! A few weeks have passed, and it is time for a new absurd exhibit: "The Risky Tube." A fragile tube filled with glowing chlorine trifluoride is locked in place while a sharp, dangerous metal ring passes back and forth along it. Using your trusty QArm once again, you must follow a specific trajectory in task space, ensuring that the ring never makes contact with the tube—lest it shatter, incinerating everyone and everything in the room. No pressure.

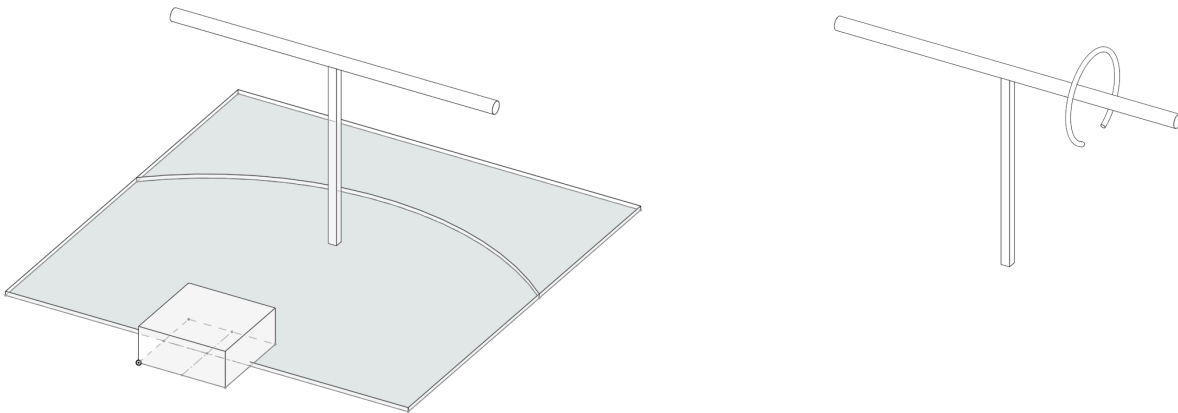


Figure 6: Environment Setup (left), ring clearance over bar (right)

A key component you will interact with is the `QArm_Lab_Interface` class, detailed below:

Method	Description	Inputs	Outputs
<code>write_to_arm(phi)</code>	Writes joint commands to the arm (in radians).	<code>phi</code>	-
<code>read_from_arm()</code>	Reads the current joint positions from the arm (in radians).	-	<code>phi</code>
<code>forward_kinematics(phi)</code>	Computes forward kinematics.	<code>phi</code>	<code>p4</code> (vector from base to EE in $\{0\}$ frame), <code>R4</code> (rotation matrix from $\{4\}$ to $\{0\}$ frame)
<code>Jacobian(phi)</code>	Computes the analytic 4×4 Jacobian.	<code>phi</code>	<code>J</code> (Jacobian matrix)

Exercise 6: A trajectory with defined knot points is provided in `trajectory_generator.py`. Implement trajectory generation using the provided Jacobian generator function and `scipy.interpolate.CubicSpline`. The knot points are formatted as 2D NumPy arrays, where each sub-array contains three values representing the (x, y, z) coordinates expressed in the base $\{0\}$ frame and a time stamp.

Exercise 7: While cubic spline trajectories are convenient and straightforward to implement, what are their main mechanical and kinematic drawbacks?

8.1.3 Exploring Integration Error

In an ideal scenario, integrating the Jacobian relationship would allow us to generate task-space trajectories continuously without ever needing to compute inverse kinematics—a significant computational advantage. Unfortunately, due to the discrete nature of digital computers and finite precision limits, each numerical iteration introduces truncation and round-off errors. Over time, this integration error accumulates to the point where the actual joint positions drift significantly from their theoretical targets, requiring external correction or recalibration.

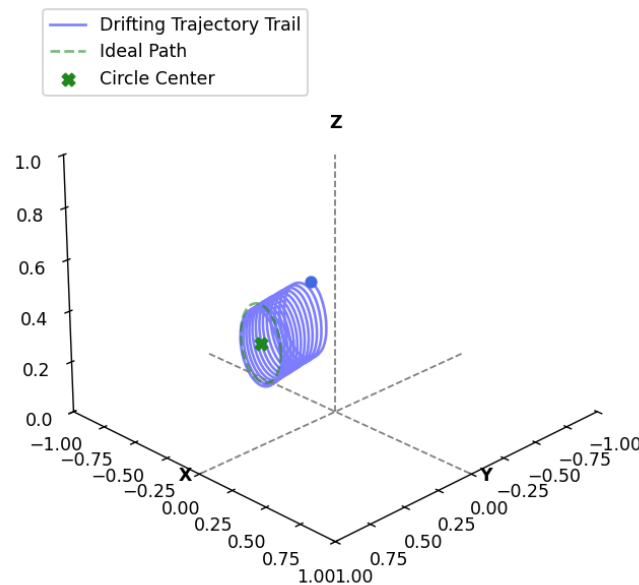


Figure 7: Accumulated integration error during a cyclic trajectory.

⚠ WARNING: Although the script is designed to terminate automatically if the end-effector drifts more than 0.4 meters away from the center of the circle, numerical drift can cause erratic robotic movements.

Keep the manipulator’s workspace entirely clear and be prepared to terminate the script immediately if required.

Run the `integration_err.py` script to observe this phenomenon. As shown in Figure 7, the arm will attempt to continuously trace a circular path in the air. Observe how the trajectory drifts over successive cycles.

LEARNING GOALS

Forward Kinematics

- ✓ *Spatial Mapping Principles*: Discover how geometric constraints reduce 3D spatial transformations down to the four unique parameters of the Denavit-Hartenberg convention.
- ✓ *Homogeneous Representations*: Grasp how 4×4 homogenous transformation matrices unify rotation and translation to seamlessly map configuration space to task space.
- ✓ *Hardware Boundaries*: Appreciate the physical and software limits of robotic manipulators, including joint limits and workspace restriction volumes.

Velocity Kinematics

- ✓ *Differential Kinematics*: Observe the role of the Jacobian and its inverse in mapping rates of change between joint variables and end-effector motion.
- ✓ *Limitations of Numerical Integration*: Recognize why digital systems suffer from truncation and round-off error, and how cumulative drift compromises open-loop control.
- ✓ *Continuity in Motion Planning*: Evaluate how boundary conditions shape trajectory profiles, and understand why continuity in lower-order derivatives (position, velocity) can still result in harsh higher-order discontinuities (jerk).

9 Extension: Workspace Boundary Defense

In the main lab, you assumed your trajectory files would always contain valid, reachable coordinate pathways. However, in real-world applications, anything from choosing two bad points to sensor error can cause the trajectory to pass into or through unreachable spaces. For a manipulator like the QArm, unreachability happens in two primary ways:

- The Outer Bound: Attempting to stretch farther than the maximum physical link reach.
- The Inner Bound: Attempting to pass too close to the central base trunk, where the mechanical links cannot physically fold inward any further.

Because our controller relies on open-loop numerical integration through the inverse Jacobian, driving the arm into these geometric boundaries causes the underlying mathematics to diverge. This can lead to severe joint velocity spikes, actuator lock-ups, or erratic physical behavior.

9.1 The Challenge

Open `lab_1_extension.py`. This script attempts to drive the end-effector sequentially through a series of targets. However, the path between two of these points forces the arm to try and cut straight through its own inner unreachable base volume. Your mission is to implement an algorithmic “Workspace Boundary Defense” mechanism directly inside your trajectory loop. You must detect the boundary violation and alter the system’s behavior dynamically.

9.1.1 Constraints

- No Hardcoding: You are strictly forbidden from hardcoding specific “detour” coordinates (e.g., `if radius < 0.1: go_to(0.2, 0.2, 0.4)`). Your solution must scale algorithmically.

9.1.2 Completion Levels

- **Level 1: Detect and Stop** Monitor the proximity to the boundary using the Jacobian’s condition number (`np.linalg.cond(J_pos)`) or forward kinematics geometry. Before the arm hits the true mechanical lock-up point, stop the movement. Print a clear warning to the console.

- **Level 2: Detect and detour** Instead of just stopping, dynamically alter the trajectory. When an inner or outer boundary violation is detected, adjust the trajectory such that the destination point is still reached.

9.2 Tasks

1. Implement: Modify the loop in `lab_1_extension.py` using your chosen strategy so that the script executes safely.
2. Demonstrate: Record a run of your implementation. Show how your system handles the unsafe boundary entry in real time.

10 Submission

Make sure to show the TA everything marked as such.

On Canvas:

- Submit your `forward_kin` function as `forward_kin.py`.
- Submit your `differential_kin` function as `differential_kin.py`
- Submit the filled worksheet
- If a solution for the extension is completed, submit it as `lab_1_extension.py` and submit a screen recording of it working.